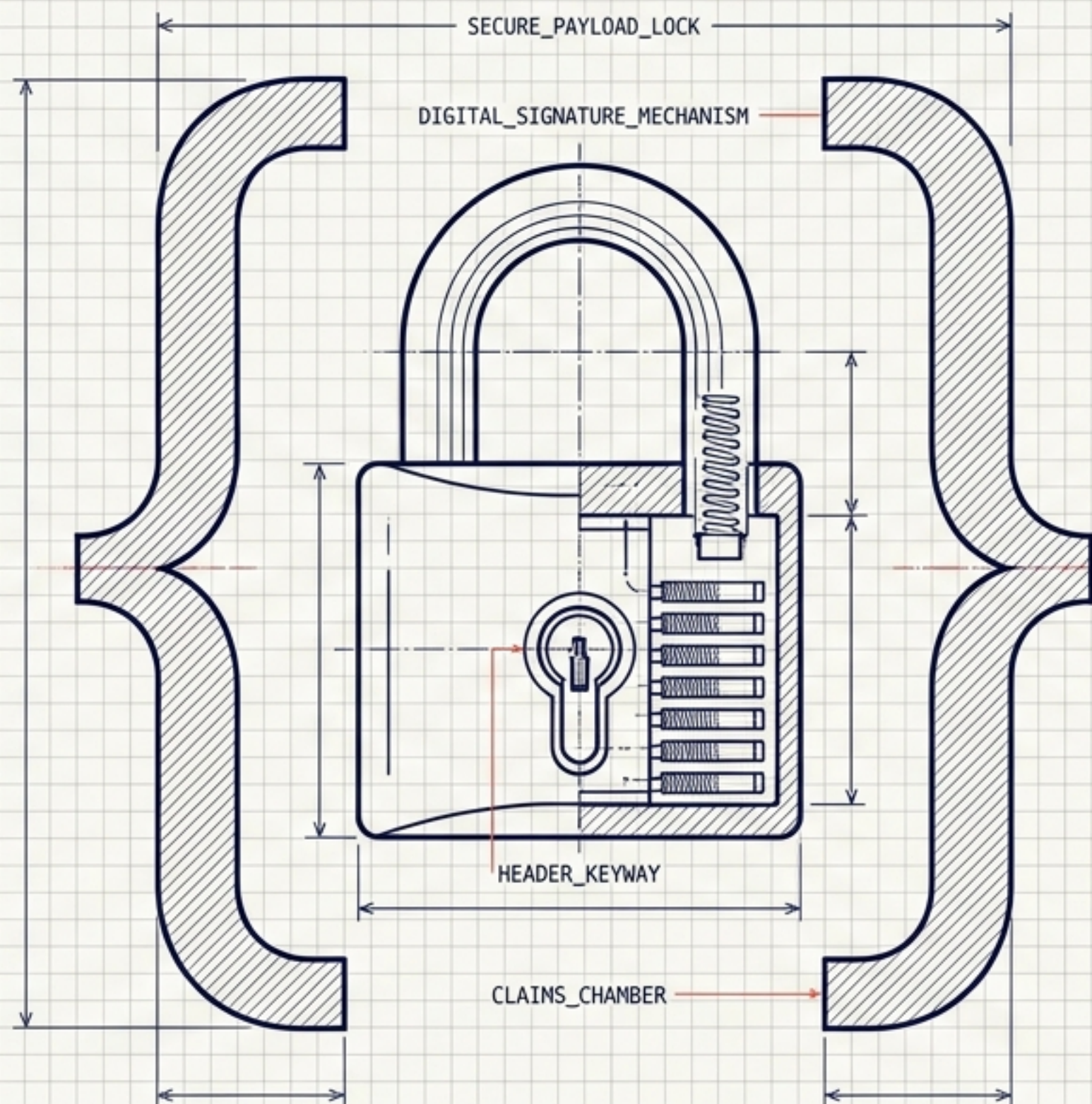


Desvendando os JWTs

Arquitetura,
Implementação e
Engenharia de
Segurança

Público-alvo:

Desenvolvedores, Arquitetos e
Engenheiros de Segurança





JWT_COMPONENT_1:
HEADER/PAYLOAD

JWT_COMPONENT_2:
SIGNATURE

JWT_COMPONENT_3:
SIGNATURE



Stateless
(Sem Estado)

Validação local. O servidor não precisa consultar o banco de dados ou um cofre de sessões para verificar o usuário.

LOCAL_VALIDATION_PROCEDURE = TRUE;
DB_QUERY = FALSE;

TOKEN_RECEIVED → VALIDATE_SIGNATURE → ~~DATABASE_LOOKUP~~



Portátil

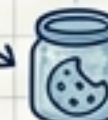
Codificação Base64URL. Seguro para trânsito via URLs, headers HTTP e cookies.

ENCODING = 'Base64URL';
SAFE_TRANSPORT_VIA = ['URL', 'HTTP_HEADER', 'COOKIE'];



Base64URL
Encode

Base64URL String



Assinado
(Integridade)

Prova matemática de que os dados não foram alterados. **Não confunda com criptografia** (ocultação de dados).

SIGNATURE_VERIFICATION = INTEGRITY_PROOF;
ENCRYPTION = FALSE;
DATA_VISIBILITY = TRUE;



eyJhbGciOiJIUzI1Ni.eyJzdWliOiIxMjM0NSIsInJvbdee-Ke6RzR0GavaLNRZ
InR5cCI6IkpXVCJ9.eyJmV4cCI6MTczNTY4OTYwMH0.Dfe8Zbk9L2c7yaqKME

O Cabeçalho (Header)

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

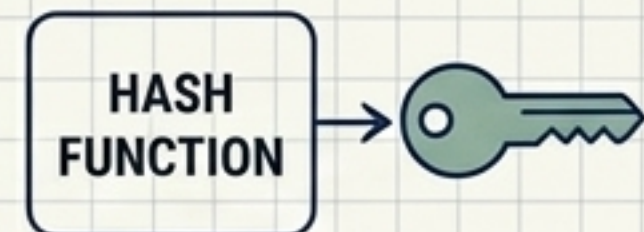
Define o tipo de token e o algoritmo criptográfico.

0 Corpo (Payload / Claims)

```
{
  "sub": "12345",
  "role": "admin",
  "exp": 1735689600
}
```

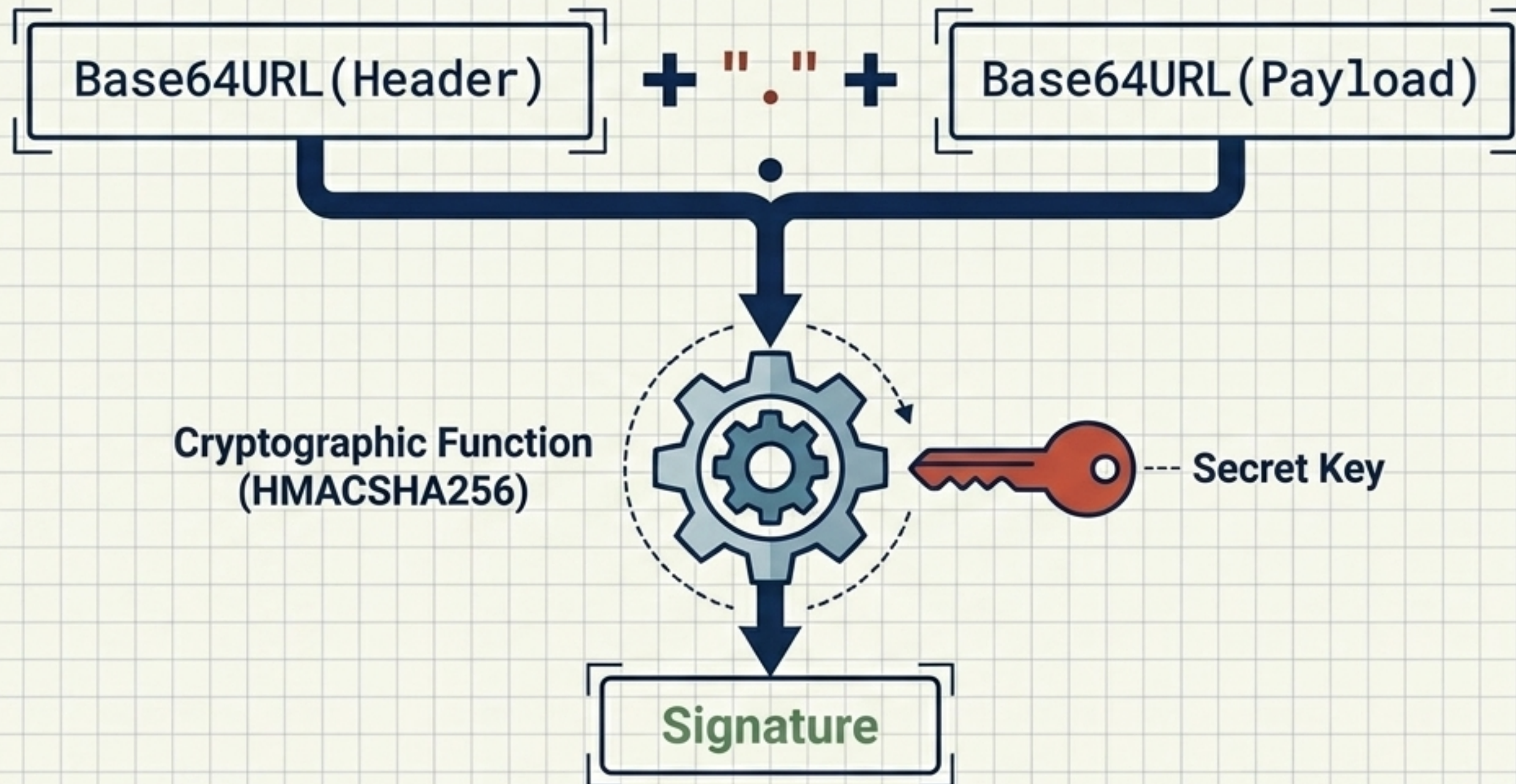
Aviso: É apenas codificado, **não criptografado**. Qualquer pessoa com o token pode ler o conteúdo.

A Assinatura (Signature)



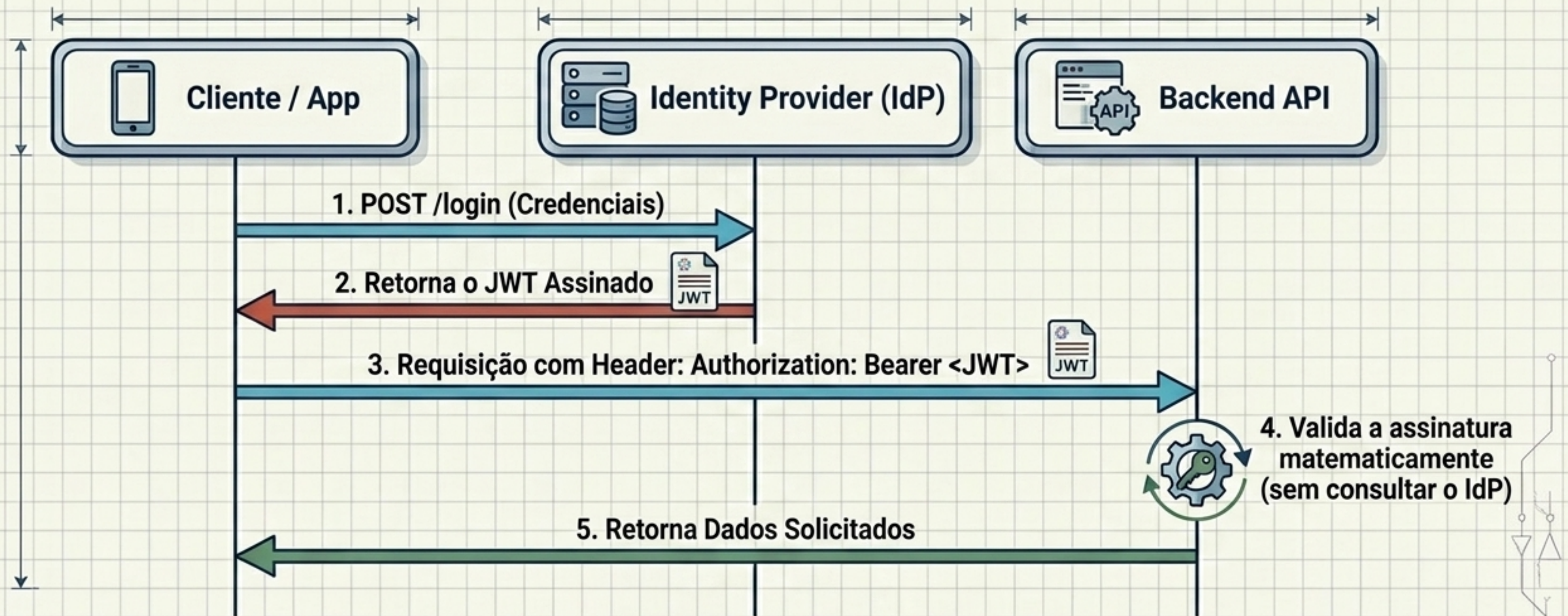
O selo de integridade gerado pelo algoritmo. Garante que o payload não foi adulterado.

O Motor Criptográfico: A Assinatura



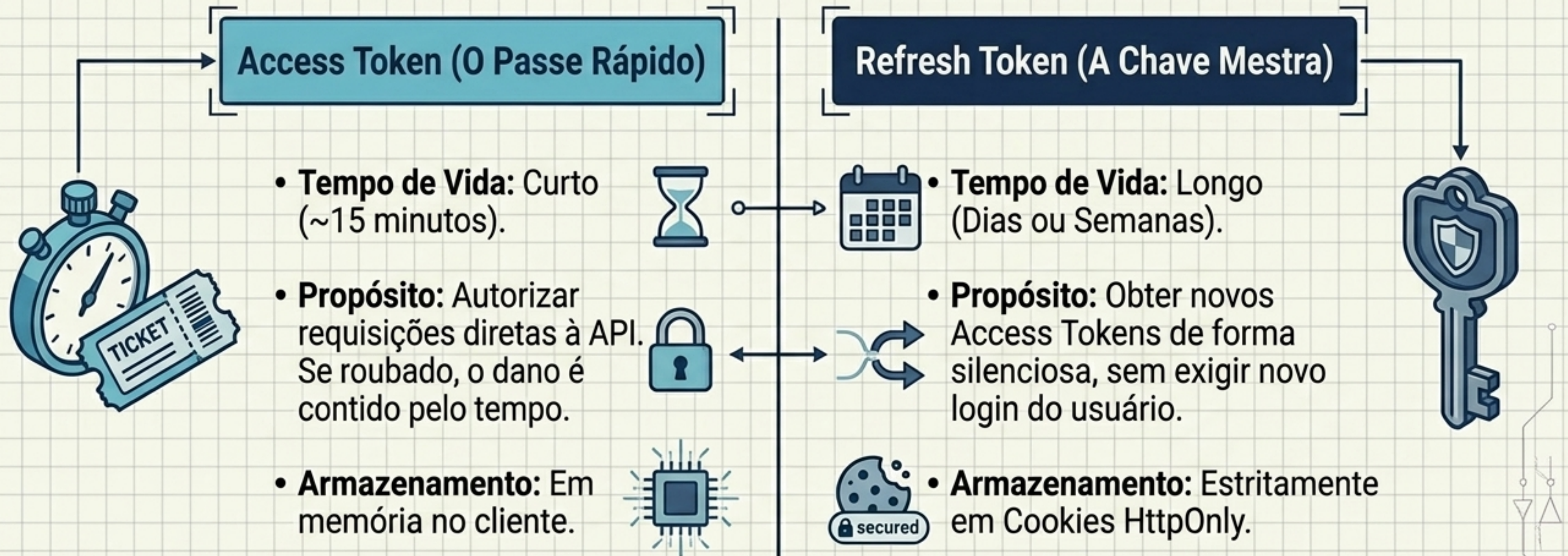
Insight Crítico: Se um atacante alterar um único caractere no Payload (ex: de "user" para "admin"), a fórmula matemática falhará durante a verificação e o token será imediatamente rejeitado.

O Fluxo de Autenticação Stateless



A grande vantagem: A Backend API opera de forma independente e não precisa de um banco de dados de sessões. Ela confia no JWT pois possui a chave criptográfica para verificar a assinatura do IdP.

A Arquitetura de Token Duplo

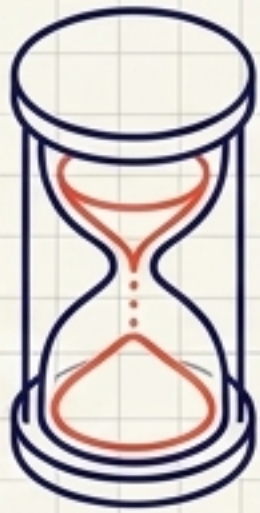


O Equilíbrio Perfeito: Combina a máxima segurança de credenciais efêmeras com a conveniência de sessões ininterruptas.

O Desafio da Revogação

Como os tokens são stateless, invalidá-los antes da expiração requer padrões de arquitetura.

1. TTL Curto (O Padrão)



Mecanismo:
Deixe o JWT expirar naturalmente.

Prática:
Access Tokens duram apenas 5-10 minutos. Revoga-se apenas o Refresh Token no servidor.

2. Deny List via Eventos



Mecanismo:
Webhooks avisam as APIs que um token foi invalidado.

Prática:
A API checa um cache local super-rápido (ex: Redis) antes de aceitar o JWT.

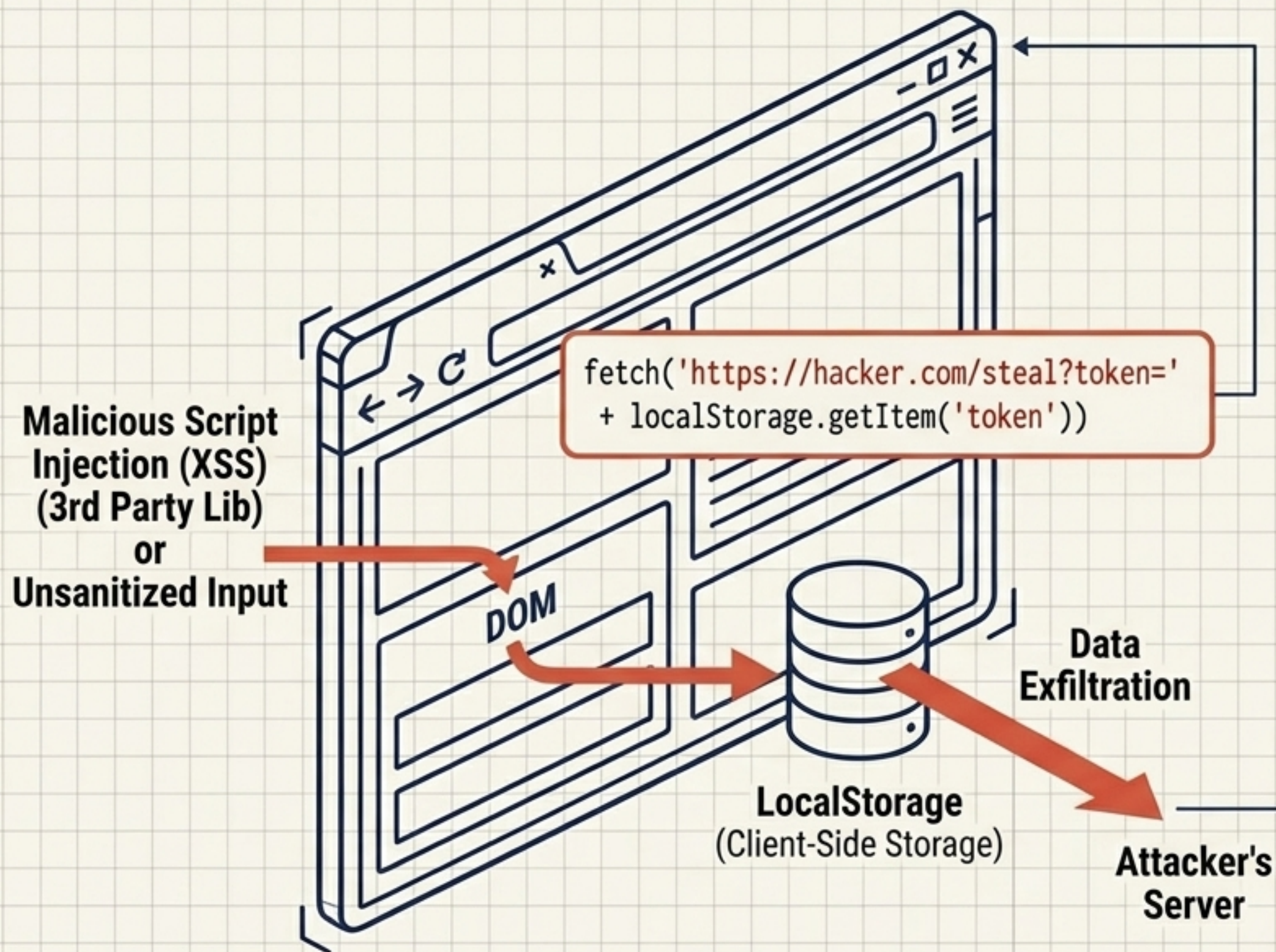
3. Rotação de Chaves



Mecanismo:
O IdP troca a chave de assinatura.

Prática:
Invalida instantaneamente TODOS os tokens assinados pela chave antiga. Drástico, mas essencial em violações.

Ameaça Crítica: A Armadilha do LocalStorage



O Problema:

O `localStorage` é globalmente acessível por qualquer código JavaScript executado na página.

A Ameaça (XSS):

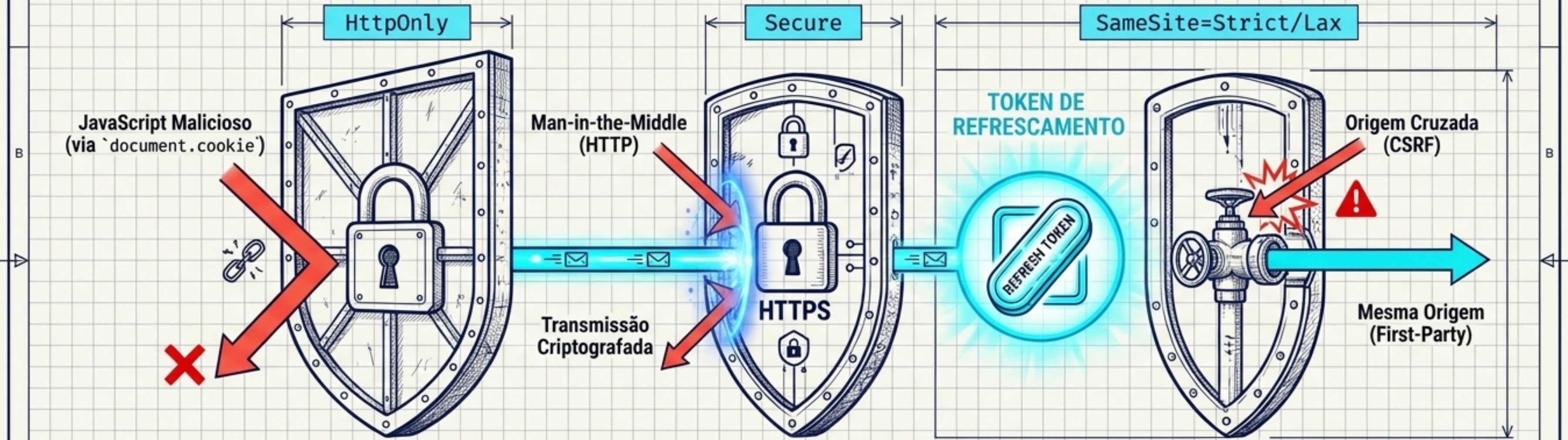
Se um script malicioso for injetado (via bibliotecas de terceiros ou falha de sanitização), o atacante rouba o token com uma única linha de código.

O Resultado:

O atacante *é* o usuário até o token expirar. O sequestro de sessão é imediato, total e silencioso.

O Padrão Ouro de Armazenamento

Mova o Refresh Token para as sombras do navegador.



O Escudo Anti-XSS.
O browser bloqueia o acesso ao cookie via `document.cookie`.
Torna o token invisível para o JavaScript malicioso.

O Escudo Anti-Interceptação.
Garante a transmissão exclusivamente sobre canais criptografados (HTTPS),
protegendo contra man-in-the-middle.

O Escudo Anti-CSRF.
Instrui o browser a nunca enviar o cookie de forma automática em requisições iniciadas por origens cruzadas (outros domínios).

		PROJECT: SECURE AUTHENTICATION	
		DRAWING: HTTP COOKIE VAULT	
		ENGINEER: SYSTEM ARCHITECT	REV. 9/7

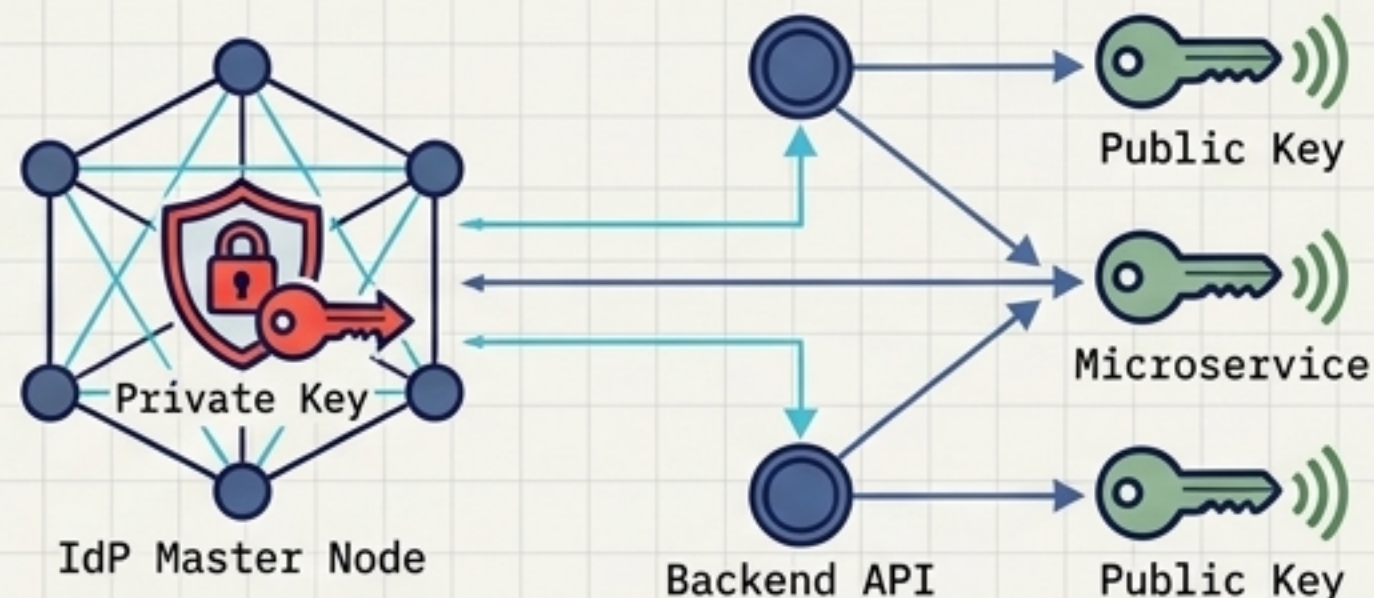
Engenharia Criptográfica: Simétrico vs. Assimétrico

Simétrico (ex: HS256)



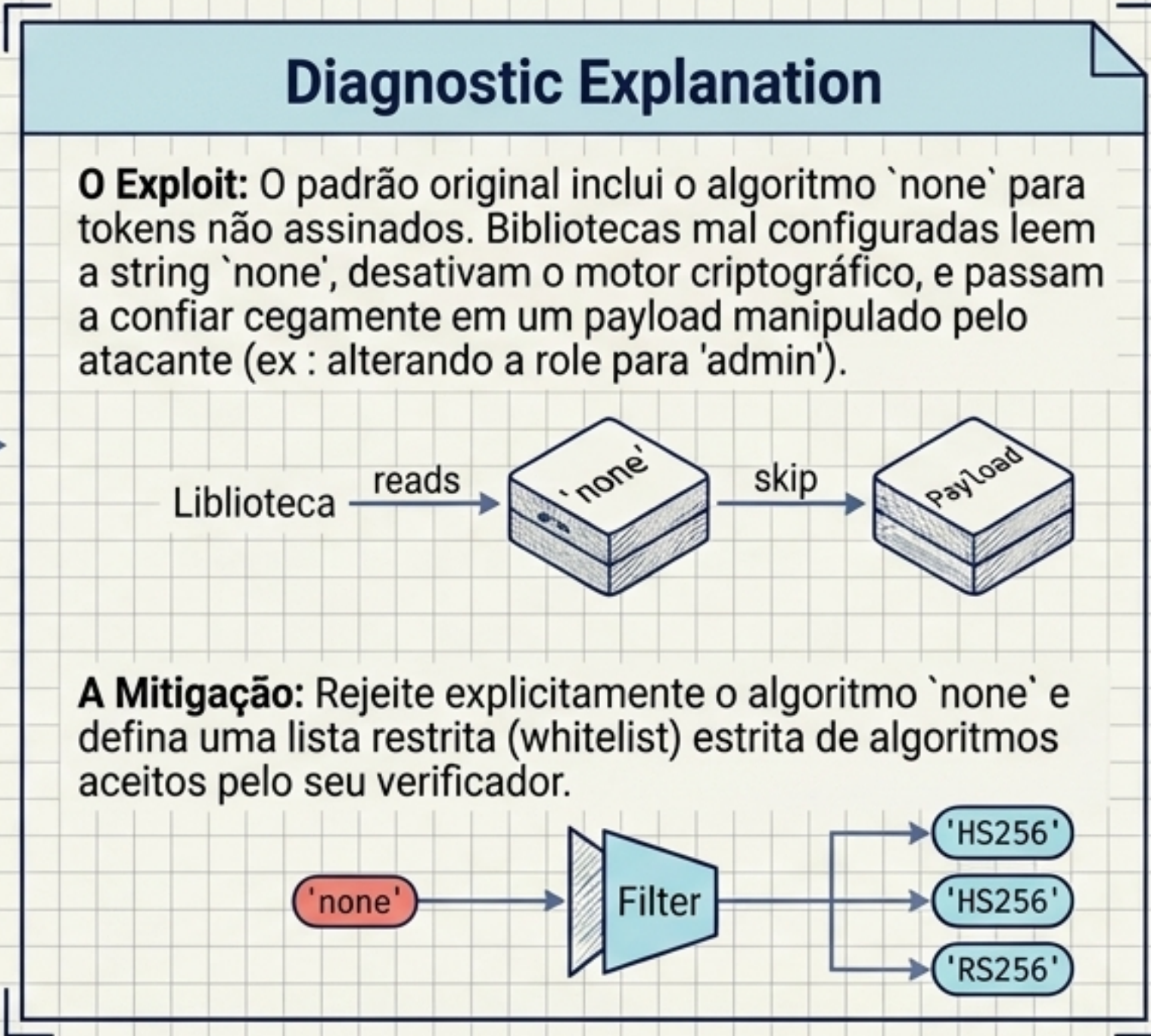
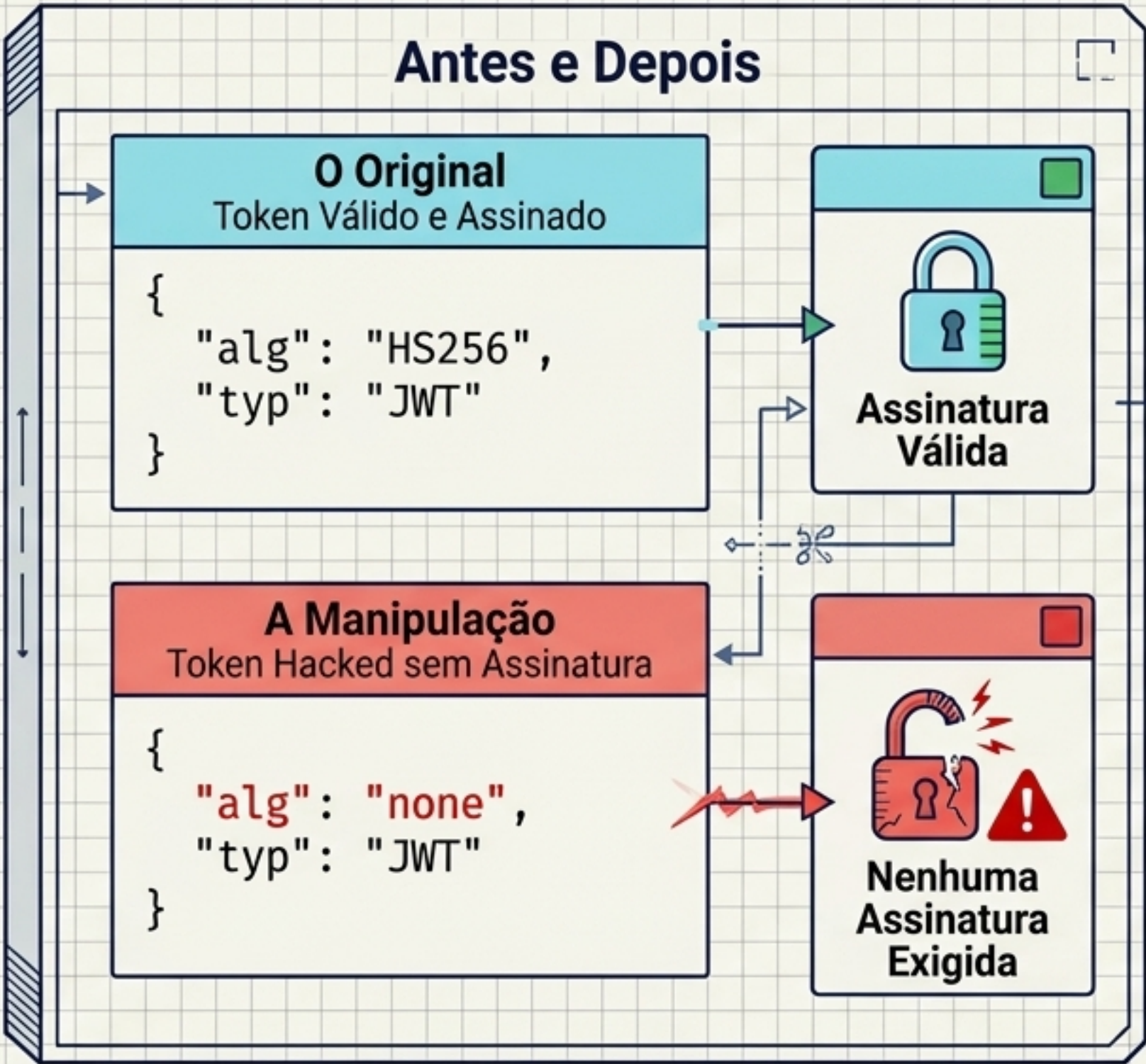
- **Mecanismo:** Usa um único **Shared Secret** para assinar e verificar.
- **Performance:** Extremamente rápido (ex: 150.000 operações/segundo).
- ⚠ **Risco Crítico:** Quem verifica também pode falsificar tokens. Exige confiança absoluta entre todos os microserviços.

Assimétrico (ex: RS256 / ES256)



- **Mecanismo:** Chave Privada no IdP assina; Chaves Públicas nas APIs verificam.
- **Performance:** Mais lento (RSA processa ~6.000 operações/segundo).
- ✓ **Vantagem Operacional:** Arquitetura trustless ideal para sistemas distribuídos. A Backend API só precisa da chave pública e jamais pode falsificar tokens.

Ponto Cego 1: O Ataque do Algoritmo 'None'



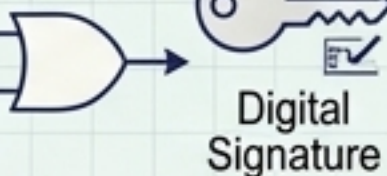
		PROJECT: SECURE AUTHENTICATION	
		DRAWING: ALGORITHM VULNERABILITY	
		ENGINEER: SYSTEM ARCHITECT	REV. 9/7

Ponto Cego 2: Confusão de Algoritmos (RS256 → HS256)

Step 1 (Server's Expectation)



Server Logic: Espera "RS256" (Assimétrico). Usa a Chave Pública para tentar verificar a assinatura.



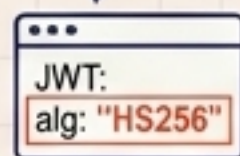
Digital Signature

Expects Asymmetric Verification

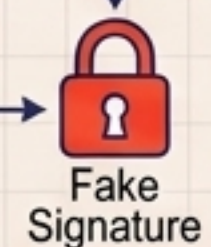
Step 2 (Attacker's Move)



Hacker Logic: O atacante altera o header do JWT para 'HS256' (Simétrico). Em seguida, ele assina o token falsificado usando a própria Chave Pública do servidor (que é pública e fácil de obter) como se fosse uma senha secreta.



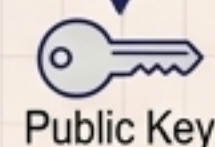
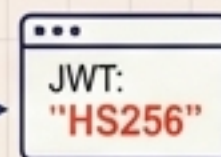
Public Key



Fake Signature

Step 3 (The Breakdown)

Server Failure: O servidor lê o header "HS256", trata sua própria Chave Pública local como o "Secret" esperado por um algoritmo simétrico, e a matemática bate perfeitamente. O token malicioso é aprovado.



Public Key



Shared Secret

Token Malicioso Aprovado

A Mitigação

```
// Sempre exija o algoritmo correto na verificação  
jwt.verify(token, publicKey, { algorithms: ['RS256'] })
```

Enforce Strict Algorithm Verification

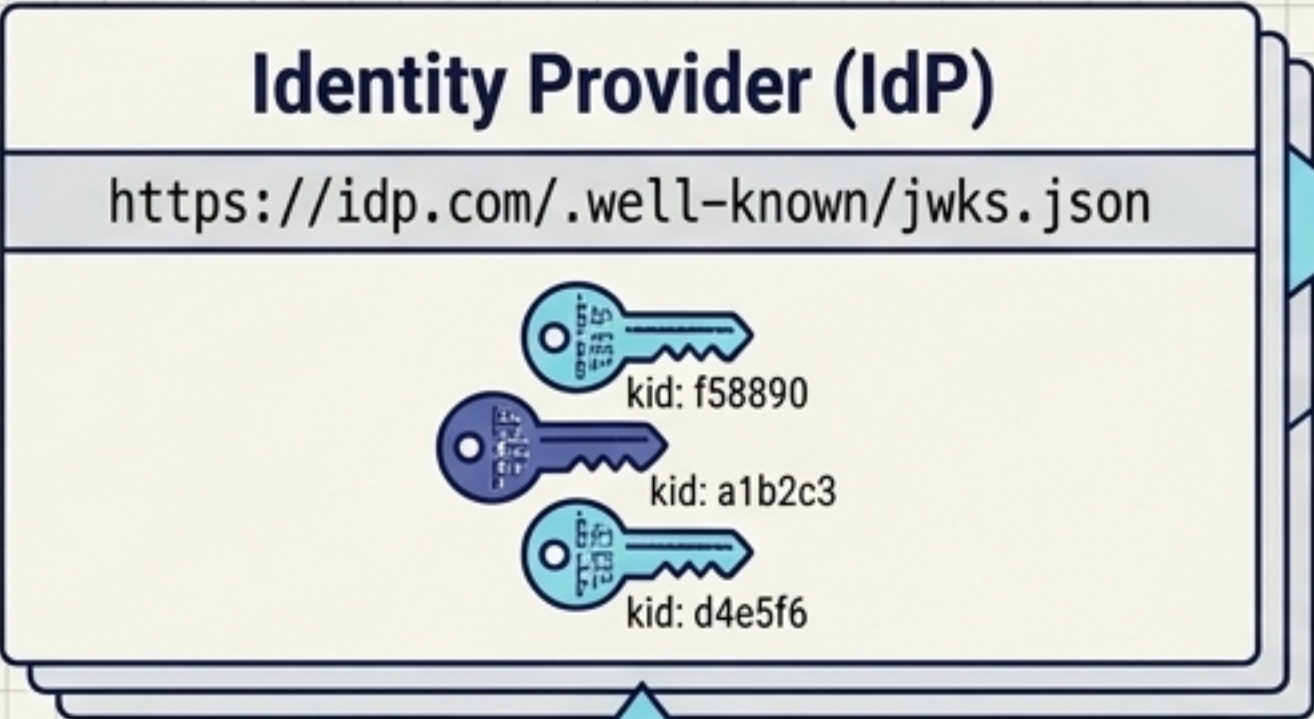
PROJECT: SECURE AUTHENTICATION

DRAWING: ALGORITHM VULNERABILITY

ENGINEER: SYSTEM ARCHITECT

REV. 9/7

Maturidade Operacional: Rotação via JWKS



JSON Web Key Set (JWKS)

O endpoint público que permite **rotação fluida** de chaves sem downtime.

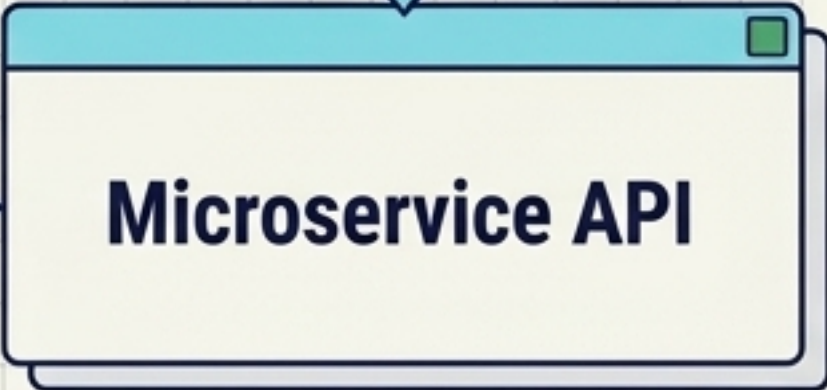
Se uma chave for comprometida, o IdP a remove do JWKS, e todos os tokens assinados por ela são instantaneamente invalidados em todo o ecossistema distribuído.

`{"kid": "f58890"}`

Busca a Chave Pública correspondente ao Key ID (kid).



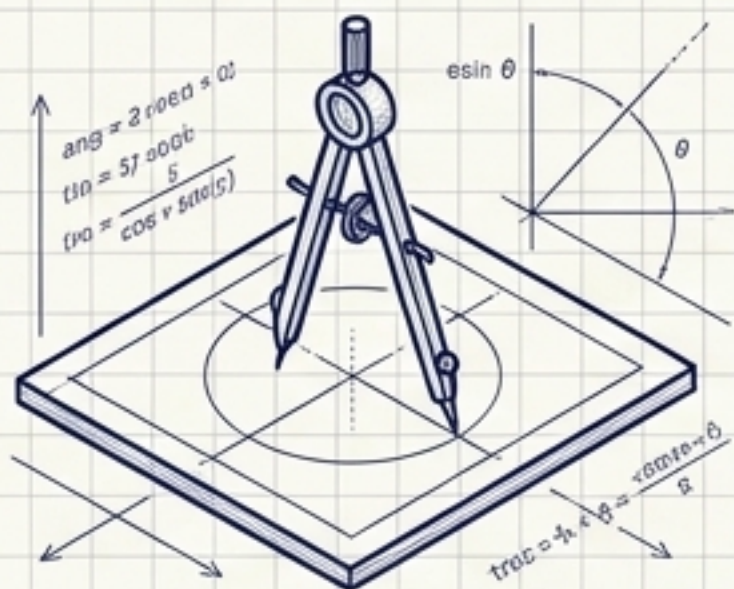
JWT



						PROJECT:	SECURE AUTHENTICATION
						DRAWING:	KEY ROTATION ARCHITECTURE
						ENGINEER:	SYSTEM ARCHITECT
						REV.	9/7

A Diferença Crítica: Verificação vs. Validação

Verificação (A Matemática)



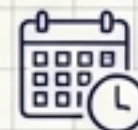
- Confirma a integridade e a autenticidade criptográfica.
- Assegura que o token foi assinado pela chave correta (Secret ou Public Key).
- Prova matematicamente que o Payload e o Header não sofreram nenhuma adulteração (tampering) em trânsito.



Validação (O Negócio)



- Confirma se as declarações (claims) fazem sentido para a regra de negócio.
- **exp**: O token já passou da data de expiração?
- **iss**: O emissor deste token é quem eu espero que seja?
- **aud**: Este token foi especificamente emitido para o meu sistema?



Regra de Ouro: Nunca valide as regras de negócio antes de verificar a matemática.

0 Checklist Definitivo de Segurança JWT

- ☐ **Algoritmos:** Faça whitelist estrito apenas dos algoritmos esperados (ex: RS256). Rejeite sempre o algoritmo none.
- ☐ **Segredos Fortes:** Em arquiteturas distribuídas, prefira chaves assimétricas via JWKS. Se usar HS256, use segredos de alta entropia.
- ☐ **Storage Seguro:** Mantenha Access Tokens efêmeros na memória. Envie Refresh Tokens apenas em cookies HttpOnly, Secure e SameSite. Evite LocalStorage.
- ☐ **Claims Padrão:** Sempre valide rigorosamente exp (Expiração), iss (Emissor) e aud (Audiência).
- ☐ **Privacidade de Dados:** Nunca inclua PII (Dados Pessoais) ou senhas no Payload. Lembre-se: O JWT é legível por qualquer um.
- ☐ **Arquitetura:** Mantenha o Access Token com tempo de vida muito curto (menor que 15 minutos).



Implemente a arquitetura, feche as brechas, escale com segurança.

PROJECT: SECURE AUTHENTICATION

DRAWING: JWT SECURITY CHECKLIST

ENGINEER: SYSTEM ARCHITECT

REV. 9/9